1) ¿Qué métodos de pasaje de argumentos podemos utilizar en una computadora?

Vía Registros: Los parámetros se pasan a través de los registros de la CPU, es un método sencillo pero altamente limitado por el número de registros disponibles. Dado que se van a modificar los contenidos de los registros, es importante documentar los registros a usar.

Vía Memoria: Los parámetros se transfieren a través de un área definida en RAM. Es difícil de estandarizar debido a las dificultades de asignar un área de memoria. Zona de memoria que acuerdan compartir tanto el programa principal como las subrutinas.

Via Pila (STACK): Los datos se pasan a través de la pila.

Su principal ventaja es que es independiente de las zonas de memoria y los registros.

Es necesario tener un buen entendimiento de cómo funciona la pila, ya que esta será usada tanto por el usuario como por el sistema.

2) ¿Cuáles son las principales diferencias entre la invocación de una subrutina y un gestor de interrupción?

Orígen de la invocación

Subrutinas: Es síncrona y determinística. El programador escribe explícitamente una instrucción en un punto específico del código.

Gestor de interrupción: Es asíncrono. Se dispara por un evento externo al flujo del programa. Puede ocurrir en cualquier momento, entre medio de cualquier par de instrucciones.

Identificación del destino

Subrutina: La dirección de destino está especificada en la propia instrucción o en un registro. El procesador ya sabe a dónde saltar.

Gestor de interrupción: El procesador no sabe a dónde ir inicialmente. Debe realizar un ciclo de **Reconocimiento de Interrupción** para obtener un “vector” o número, y luego buscar la dirección real en una tabla en memoria.

Preservación del estado

Subrutina: Normalmente, solo se guarda automáticamente el PC en la pila. Si se necesitan salvar otros registros, lo debe hacer el programador mediante software.

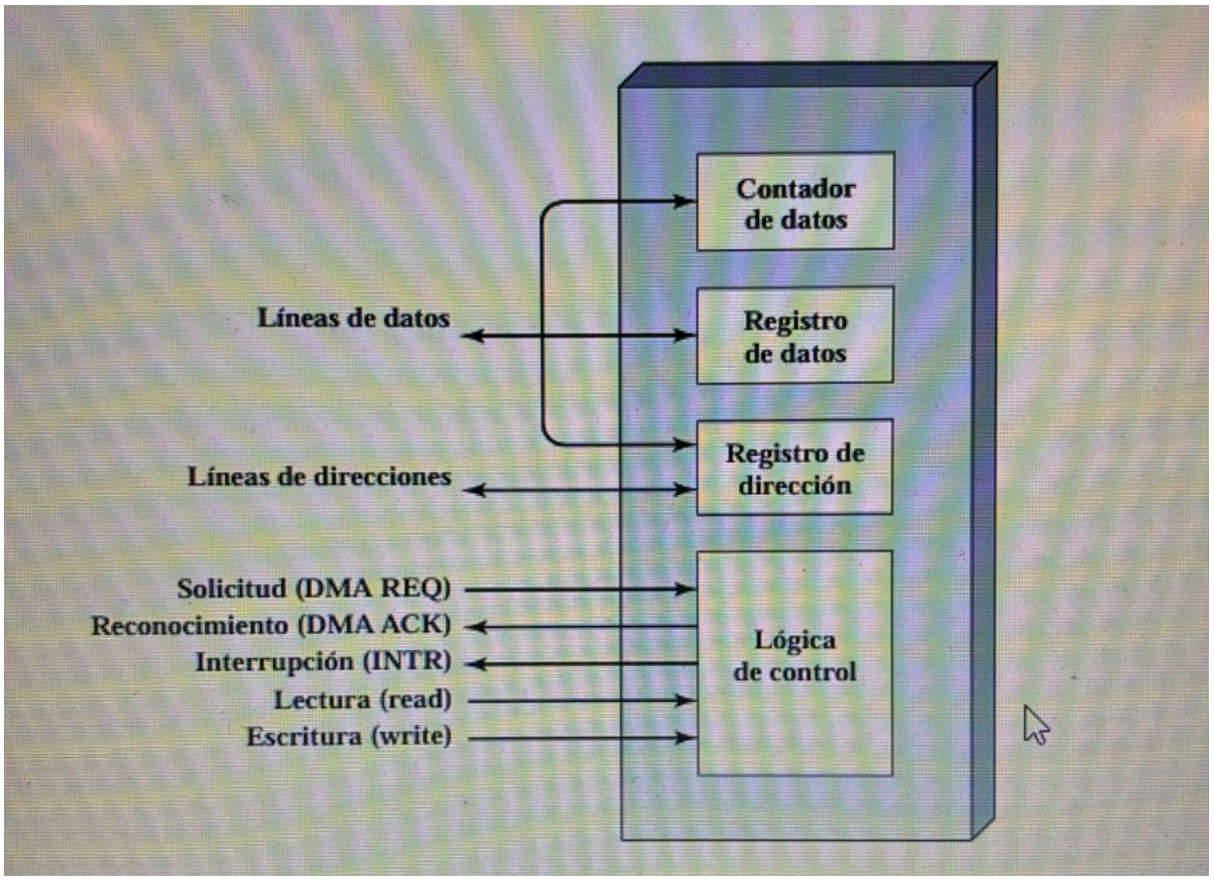
Gestor de interrupción: El hardware debe salvar automáticamente el PC y el PSW. Esto es importante porque, como la interrupción es asíncrona, no se sabe qué estaba haciendo el programa principal en ese momento, por lo tanto debe guardarse el estado del CPU para poder continuar con dicha ejecución.

Instrucción de retorno

Subrutina: Finaliza con una instrucción de retorno común (ej. RET). Solo recupera el PC de la pila.

Gestor de interrupción: Finaliza con una instrucción especial (ej. IRET). Esta instrucción recupera de la pila tanto el PC como el PSW.

3) Esquematice y describa la estructura interna del Controlador de Acceso Directo a Memoria.



Registros internos: Unidades de almacenamiento que el controlador usa para saber qué tiene que hacer:

- Contador de datos: Guarda la cantidad de datos que faltan transferir. Se decrementa con cada dato movido hasta llegar a cero.
- Registro de datos: Buffer temporal donde se mantiene el dato que se está transfiriendo en ese instante entre el periférico y la memoria.
- Registro de dirección: Almacena la dirección de memoria exacta donde se debe leer o escribir el siguiente dato. Se incrementa automáticamente tras cada transferencia exitosa.

Lógica de control y señales: “Cerebro” del controlador que gestiona el diálogo con la CPU y los periféricos:

- DMA REQ (Solicitud): Es la entrada que recibe el controlador desde un periférico indicando que tiene datos listos para transferir.
- DMA ACK (Reconocimiento): Es la respuesta del controlador al periférico confirmado que ha tomado el control y comenzará la transferencia.

- INTR (Interrupción): Es la señal que el controlador envía a la CPU una vez que el **contador de datos** llega a cero.
- Read/Write: Son las señales que indican si la operación actual es de lectura o escritura sobre la memoria o el periférico.

Conexiones externas (Buses): El controlador necesita “manejar” los buses del sistema como si fuera la propia CPU:

- Líneas de datos: Conectadas al **registro de datos** para mover el contenido real.
- Líneas de dirección: Conectadas al **registro de dirección** para indicar el destino/origen en la RAM.

Flujo completo

1. La CPU configura el controlador (dirección de inicio y cuántos datos mover).
2. El periférico activa DMA REQ.
3. El controlador le pide a la CPU acceso a los buses.
4. El controlador mueve los datos de forma autónoma, actualizando el **contador** y la **dirección**.
5. Cuando termina, envía un **INTR** a la CPU para informar el fin de la tarea.

<u>4) Describa el funcionamiento de la gestión de E/S programada con espera de respuesta</u>

La CPU interviene directamente en la transferencia de cada unidad de información con el módulo, es decir que tiene control directo sobre la operación de E/S, además, comprueba el estado del dispositivo, envía comandos de lectura/escritura y realiza la transferencia de todos los datos de a uno.

El módulo de E/S realiza la acción solicitada por el procesador, y después activa los bits apropiados en el registro de estado de E/S.

El procesador es responsable de comprobar periódicamente el estado del módulo de E/S hasta que encuentra que la operación ha terminado, provocando que permanezca “dormido”, ya que espera la respuesta.

La secuencia de acciones que ejecuta la CPU es:

1) La CPU verifica el estado del periférico leyendo un registro del módulo de interfaz.
2) Examina el estado del periférico chequeando el/los bit/bits que identifican dicho estado.
3) Si el dispositivo no está listo, el CPU vuelve al paso 1. Este lazo significa que la CPU espera hasta que el periférico esté listo para la transferencia.
4) Cuando el dispositivo está listo, el CPU transfiere un dato hacia o desde el módulo de interfaz.

5) Si hay más datos que transferir, vuelve al paso 1.

6) Si se completó la transferencia, termina el servicio de E/S.

5) ¿Cuáles son las políticas de escritura de la memoria caché?

Políticas de escritura en aciertos (HIT)

- **Escritura inmediata (Write-Through):** Todas las operaciones de escritura se hacen tanto en caché como en memoria principal, asegurando que el contenido de la memoria principal siempre es válido. Su principal desventaja es que genera un tráfico sustancial que puede generar un cuello de botella.

- **Post-Escritura (Write-Back):** La información solo se actualiza en la caché y se escribe en la memoria cuando se reemplaza el bloque. El bloque requiere de un bit "sucio" para indicar cuándo se escribió.

Políticas de escritura en fallos (MISS)

- **Asignación sin escritura (No-Write Allocate):** Se escribe directamente en la memoria principal. La caché se usa solo en lecturas. El bloque no se lleva a la memoria caché mientras no se tenga que leer. Habitualmente usado con **Escritura Inmediata**.

- **Asignación con Escritura (Write Allocate):** El bloque requerido se copia en la caché y luego se escribe (en la caché). Habitualmente usado con **Post-Escritura**.

6)¿La coherencia de datos de un sistema jerárquico de memorias se ve afectada por el uso de DMA?

Sí, el uso de DMA (Acceso Directo a Memoria) puede afectar la coherencia de datos en un sistema jerárquico de memoria. Esto se debe a que el DMA permite que dispositivos periféricos accedan directamente a la memoria principal sin pasar por la jerarquía de caché, lo que puede dar lugar a inconsistencias de datos en la caché y la memoria principal. Para mantener la coherencia de datos, se requieren técnicas especiales como **invalidación de caché** y **protocolos de coherencia de caché** para garantizar que los datos se sincronicen correctamente.

7) ¿De qué depende el paralelismo de una máquina superescalar?

Paralelismo a nivel de instrucción del programa: Factor más importante dependiente del software. Es el grado en que las instrucciones de un programa pueden ejecutarse en paralelo sin que el resultado cambie. Es una propiedad del algoritmo.

Limitaciones de Hardware: El paralelismo está limitado por cuántas unidades funcionales se hayan duplicado. Para que un procesador superescalar pueda funcionar, es necesario tener duplicación de hardware para poder ejecutar múltiples instrucciones al mismo tiempo.

Dependencias y riesgos: El paralelismo real se sigue viendo afectado por los atascos

- Dependencias de datos (RAW, WAW, WAR)

- Dependencias de Control (Saltos)

Políticas de emisión: La política que utiliza la computadora para determinar el orden de emisión y finalización. Una máquina que permite la emisión desordenada siempre logrará más paralelismo que una que emite en orden.

8) ¿Cuál es el objetivo de usar la técnica de Renombre de Registros?

El Renombre de Registros es una técnica principalmente utilizada por los procesadores superescalares (pero no es exclusiva de ellos) para gestionar el uso de mismo registros físicos en múltiples instrucciones.

Cuando más de una instrucción necesita utilizar un mismo registro lógico para hacer una operación, esto genera una antidependencia (WAR) o una dependencia de salida (WAW), por lo que se aplica la técnica de Renombre de Registros.

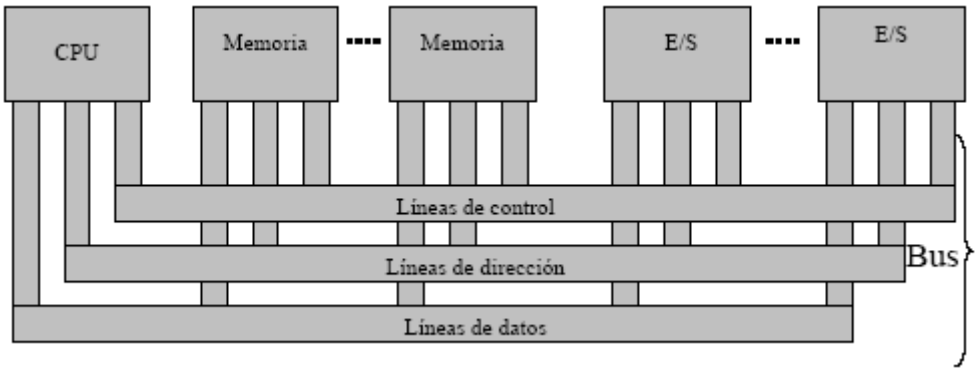
Esta técnica plantea la utilización de otro registro físico para simular el uso de los mismos nombres lógicos, lo cual permite que las instrucciones crean que pueden ejecutarse con normalidad, pero realmente están depositando o leyendo la información de un registro físico distinto que fue renombrado para hacerse pasar por el que debía ser utilizado.

9) ¿Cuáles son los elementos principales que definen un bus?

Un bus es un camino de comunicación entre dos o más dispositivos. Se trata de un medio de transmisión compartido.

Al bus se conectan varios dispositivos, y, cualquier señal transmitida por uno de esos dispositivos, está disponible para que los otros dispositivos conectados al bus puedan acceder a ella.

Un bus está constituido por varios caminos de comunicación o líneas. Cada línea es capaz de transmitir señales binarias representadas por 0 y 1. En un intervalo de tiempo, se puede transmitir una secuencia de dígitos binarios a través de una única línea.



El bus de sistema está constituido por una amplia cantidad de líneas. A cada línea se le asigna un significado o una función particular. Las líneas pueden ser de datos, de direcciones y de control.

Líneas de datos

- Proporcionan un camino para transmitir datos entre los módulos del sistema.

- El Bus de Datos tiene un ancho, el número de líneas determina cuántos bits se pueden transferir al mismo tiempo en dicho bus.
- El conjunto de estas líneas se denomina Bus de Datos.

Líneas de dirección

- Designan la fuente o el destino del dato situado en el Bus de Datos.
- La anchura del bus de direcciones determina la capacidad máxima de memoria posible en el sistema.
- Las líneas de dirección son usadas para direcciones los puertos de E/S.
- El conjunto de estas líneas se denomina Bus de Direcciones.

Líneas de control

- Controlan el acceso y el uso de las líneas de datos y dirección
- Las señales de control transmiten tanto órdenes (operaciones a realizar), como información de temporización (indican la validez de datos y direcciones) entre los módulos del sistema.
- El conjunto de estas líneas se denomina Bus de Control.

El bus que conecta los componentes principales de la computadora se denomina bus de sistema.

Tipo de líneas: Las líneas de los buses se pueden dividir en dos tipos genéricos

- Dedicado: Se refiere a un bus que está reservado exclusivamente para la comunicación entre un componente específico del sistema y otros dispositivos.
- Multiplexado: Indica si el bus multiplexa múltiples señales o canales de datos en un solo medio de transmisión.

Método de arbitraje: Se refiere al proceso utilizado para gestionar y controlar el acceso de múltiples dispositivos al bus

- Centralizado: Se refiere al enfoque de arbitraje en el que un único controlador central administra el acceso al bus.
- Distribuido: Indica un método de arbitraje en el que múltiples dispositivos compiten entre sí por el acceso al bus. Al no haber un controlador que se encarga de gestionar el acceso, todos los dispositivos están conectados a una línea de control y ponen su ID de prioridad, si en la línea hay una ID de prioridad mayor a la del dispositivo, lo deja pasar. De esta manera se asegura que el dispositivo con más prioridad sea el primero en tener acceso al bus.

Temporización: Hace referencia a la coordinación de las operaciones de transferencia de datos en el bus en relación con el tiempo

- Síncrono: Las operaciones de transferencia de datos están sincronizadas con una señal de reloj común.
- Asíncrono: Las operaciones de transferencia de datos no están sincronizadas con una señal de reloj común. Las operaciones pueden ocurrir en cualquier momento.

Tipo de transferencia: Se refiere al movimiento de información desde una ubicación de almacenamiento hacia otro componente del sistema

- Lectura: Proceso de obtener datos desde una fuente hacia un dispositivo que los solicita.
- Escritura: Proceso de enviar datos desde un dispositivo que los genera hacia un destino, como la memoria o un dispositivo periférico.
- Bloque: Transferencia de una cantidad específica de datos agrupados en una unidad llamada bloque.

<u>10) ¿Qué son los MIMD de la taxonomía de Flynn?</u>

MIMD, o Multiple Instruction, Multiple Data, es una categoría de computadoras donde existen múltiples procesadores, múltiples instrucciones y múltiples datos. Todos trabajan al mismo tiempo pero de forma totalmente independiente.

Es el modelo más exitoso debido a que es el más flexible. Al tener múltiples Unidades de Control, un sistema MIMD puede:

- Ejecutar un solo programa muy grande repartiendo tareas (**Paralelismo**)
- Ejecutar muchos programas distintos a la vez (**Multitarea**)

MIMD representa el nivel más alto de paralelismo. Permite que múltiples procesadores autónomos ejecuten sus propios hilos de instrucciones sobre sus propios datos. Se divide principalmente en sistemas que comparten una memoria común y sistemas que tienen memorias separadas y se comunican por red.

Multiprocesadores de Memoria Compartida (SMP y NUMA): Todos los procesadores están en la misma "placa" y ven la misma memoria RAM.

SMP -> Symmetric Multiprocessing: Todos los procesadores tardan lo mismo en llegar a cualquier parte de la memoria.

<u>Ventajas:</u>

- Para el programador es fácil, ya que todos los procesadores ven la misma memoria.
- El Sistema Operativo puede mover procesos de un núcleo a otro fácilmente.

<u>Desventajas:</u>

- A medida que se agregan más CPUs, el bus de datos y la memoria se saturan. Genera cuello de botella.
- Mantener coherencia de las cachés de cada CPU es muy costoso a medida que crece el sistema.

NUMA -> Non-Uniform Memory Access: Cada procesador tiene un pedazo de memoria más "cerca". Puede leer toda la RAM, pero tarda más si el dato está en la zona del "vecino".

Ventajas:

- Permite conectar decenas o cientos de procesadores sin saturar un único bus.
- El acceso a la memoria que está en el mismo nodo es extremadamente rápido.

Desventajas:

- Si una CPU necesita datos que están en la memoria de otro nodo, el acceso es mucho más lento.
- El Sistema Operativo debe ser inteligente para poner los datos cerca de la CPU que los va a usar.

Multicomputadoras de Memoria Distribuida (Clusters): Ya no se habla de un chip, sino de varias computadoras conectadas por red.

- Cada procesador tiene su propia RAM y nadie más puede verla.

- Si el procesador A necesita un dato del procesador B, tienen que mandarse un "mensaje" por la red (paquete de datos).

Ventajas:

- Podés seguir agregando computadoras a la red para aumentar la potencia.
- Si un nodo falla, los demás siguen funcionando. Es ideal para servidores críticos.
- Se puede armar un cluster potente usando hardware comercial estándar.

Desventajas:

- Los programas deben estar diseñados para dividirse y comunicarse por red (pasaje de mensajes).
- La comunicación entre nodos es órdenes de magnitud más lenta que dentro de una misma placa madre.

11) ¿Qué elementos componen una máquina con arquitectura Von Neumann? Describir la función de cada uno.

Unidad Central de Procesamiento (CPU): Cerebro del sistema, se divide en dos partes críticas:

- **Unidad de Control (UC):** Se encarga de buscar las instrucciones en la memoria, decodificarlas y generar las señales de control para que el resto de los componentes las ejecuten.

- **Unidad Aritmético Lógica (ALU):** Realiza las operaciones matemáticas y lógicas.

Memoria Principal (RAM): Espacio de almacenamiento temporal, direccionable y volátil. Almacena tanto los datos como las instrucciones del programa que se está ejecutando en ese momento.

Unidad de Entrada/Salida (I/O): Interfaz con el mundo exterior.

- **Entrada:** Permite que la computadora reciba datos.
- **Salida:** Permite que la computadora devuelva resultados al usuario.

Buses del Sistema: Son las vías por donde viaja la información entre los componentes anteriores. Se dividen según lo que transportan:

- **Bus de Datos:** Transporta la información real.
- **Bus de Direcciones:** Transporta la ubicación a la que se quiere acceder.
- **Bus de Control:** Transporta las órdenes.

12) ¿Cuáles son los elementos a tener en cuenta para el diseño de una memoria caché?

Para el diseño de la caché hay que tener en cuenta varias consideraciones:

Tamaño de caché: La caché debe ser lo suficientemente grande para contener la mayor cantidad posible de información.

Cuanto más grande es la caché, mayor es el número de puertas implicadas en direccionar. El resultado es que las memorias caché más grandes tienden a ser ligeramente más lentas que las pequeñas.

El tamaño de las cachés está también limitado por las superficies disponibles de chip y de tarjeta que contiene el procesador.

Tamaño de línea/bloque: A medida que aumenta el tamaño de bloque, la tasa de aciertos primera aumenta debido al principio de localidad. Sin embargo, la tasa de aciertos comenzará a decrecer cuando el tamaño de bloque se haga aún mayor y la probabilidad de utilizar la nueva información captada se haga menor. Cuanto más grande es el bloque, más se reduce el número que cabe en la caché.

A medida que un bloque se hace más grande, cada palabra adicional está más lejos de la requerida, y, por tanto, es más improbable que sea necesaria a corto plazo.

Número de cachés: La memoria caché puede ser una sola o estar dividida en varias unidades (múltiples niveles). Los diseños más actuales incluyen tanto una caché on-chip como otra externa, resultando en una caché de dos niveles, siendo la caché interna el nivel 1 (L1) y la externa el nivel 2 (L2).

Esta idea de mantener las memorias caché separadas permite tener distintos tamaños, políticas de acceso y reemplazo, además, incrementa la cantidad de aciertos ya que, si el dato se encuentra en cualquier nivel de la caché, no será necesario ir a buscarlo a la memoria principal.

Función de correspondencia: La función de correspondencia determina cómo se organiza la caché, es decir, la forma en la que se van a asignar los bloques de la memoria principal en la memoria caché.

- Correspondencia directa: Consiste en hacer corresponder cada bloque de memoria principal a solo una línea posible de la caché. Esto se implementa utilizando la

dirección. Cada dirección de memoria principal puede verse como dividida en tres campos:

- Tag: Número de grupo en la memoria principal.
- Line: Número de bloque dentro del grupo.
- Word: Número de palabra dentro del bloque.

Esto proporciona una correspondencia única o asignación única de cada bloque de memoria principal en la caché, lo cual quiere decir que solo existe una posibilidad donde el bloque puede existir en esta.

Su principal desventaja es una baja tasa de aciertos y grandes pérdidas de caché, debido a que hay solo una posición concreta de caché para cada bloque dado, si un programa accede a palabras de dos bloques diferentes en la misma línea de la caché, se intercambiarán continuamente.

- Correspondencia asociativa: Permite que cada bloque de memoria principal pueda cargarse en cualquier línea de la caché.
- Correspondencia asociativa por bloques: Un bloque puede almacenarse en un conjunto restringido de lugares en la caché.

  La caché se divide en un número de conjuntos N, cada conjunto N contiene un número de líneas y un bloque corresponderá a alguna línea de un conjunto determinado. Combina lo mejor de las otras correspondencias.

Algoritmos de sustitución: Cuando hay un fallo de acceso a la memoria caché, se debe traer un bloque desde la memoria principal y almacenar un bloque existente en la memoria caché. El lugar a donde va a ser ubicado el bloque a traer desde la memoria requiere reemplazar un bloque existente.

- Correspondencia directa: Al haber solo una posible línea para bloque particular, no hay elección posible. Por lo tanto, si se debe traer un bloque desde la memoria principal, el bloque de la caché será reemplazado inevitablemente.
- Correspondencias asociativas (ambas): Existen algoritmos de sustitución que se implementan a nivel hardware:
  - Least Recently Used (LRU): Se sustituye el bloque que se ha mantenido en la caché por más tiempo sin haber sido referenciado. Requiere controles de tiempos y aprovecha la localidad temporal.
  - First In, First Out (FIFO): Se sustituye el bloque del conjunto que ha estado más tiempo en la caché. Requiere controles de acceso para identificar el orden en el que ingresaron.
  - Least Frequently Used (LFU): Se susituye el bloque del conjunto que ha experimentado menos referencias. Se puede implementar asociando un contador a cada línea.

- Aleatoria: Consiste en reemplazar una línea al azar entre las posibles candidatas.

Política de escritura: Cuando la CPU guarda un resultado en la memoria, puede hacerlo si la dirección está en un bloque de la caché (HIT) o si no lo está (MISS). En ambas situaciones, desde el punto de vista de la coherencia de datos, se debe evitar inconsistencias de información entre la memoria principal y la caché durante el proceso de escritura.

Políticas de escritura en aciertos:

- Escritura inmediata (write-through): Todas las operaciones de escritura se hacen tanto en la caché como en la memoria principal, asegurando que el contenido de la memoria principal siempre es válido.

- Post-escritura (write-back): La información solo se actualiza en la caché y se escribe en la memoria cuando se reemplaza el bloque. El bloque requiere de un bit “sucio” para indicar cuándo se escribió.

Políticas de escritura en fallos:

- Asignación sin escritura (no-write allocate): Se escribe directamente en la memoria principal. La caché se usa solo en lecturas. El bloque no se lleva a la memoria caché mientras no se tenga que leer.

- Asignación con escritura (write allocate): El bloque requerido primero se copia en la caché, y luego se escribe.

Asignación sin escritura típicamente se utiliza con **Escritura inmediata**.

Asignación con escritura típicamente se utiliza con **Post-escritura**

13) Segmentación de cauce: describa los métodos y técnicas utilizadas para disminuir o evitar las paradas que afectarán al funcionamiento de los cauces.

Técnicas para Dependencias de Datos: El objetivo es que la instrucción siguiente no tenga que esperar a que el dato se escriba definitivamente en el banco de registros.

- Forwarding: Técnica de hardware muy efectiva. Consiste en añadir caminos internos (cables y multiplexores) que toman el resultado de la ALU y lo envían directamente a la entrada de la ALU para la siguiente instrucción, sin pasar por los registros.

- Renombramiento de Registros: Permite eliminar las “falsas” dependencias (WAR y WAW) utilizando un conjunto de registros físicos adicionales. Esto evita que múltiples instrucciones compitan por el mismo nombre de registro lógico, permitiendo que instrucciones independientes se ejecuten en paralelo aunque el código original use el mismo nombre de registro.

- Reordenamiento de código: El compilador detecta la dependencia y busca una instrucción independiente que esté más adelante para “rellenar” el hueco mientras el dato se calcula.

Técnicas para Dependencias de Control: Debido a que los saltos rompen la secuencialidad, se busca predecir el destino para no vaciar el cauce.

- Predicción de Saltos (Branch Prediction):
    - Estática: Se asume siempre una dirección (por ej. los saltos en bucles siempre se toman).
    - Dinámica: El hardware tiene una tabla “BHT” (Branch History Table) que almacena si ese salto se tomó o no en el pasado. Si acierta, la penalización es cero.
- Salto Retardado (Delayed Branch): El compilador coloca una instrucción útil inmediatamente después del salto. Esa instrucción se ejecuta siempre, aprovechando el ciclo.
- Buffer de Destino de Salto (BTB): Una caché que guarda la dirección de la instrucción de salto y su dirección de destino ya calculada. Además, guarda un historial para indicar si el salto se tomó o no en veces anteriores. Esto permite que el procesador empiece a captar instrucciones del destino incluso antes de que la instrucción de salto termine de decodificarse.

Técnicas para Dependencias Estructurales: La única solución es la duplicación de recursos.

- Arquitectura Harvard (Cachés separadas): Para evitar que la búsqueda de instrucción (IF) y el acceso a memoria (MEM) choquen, se utilizan cachés físicas distintas para instrucción y para datos. De esta manera, ambos pueden acceder a su propia memoria en el mismo ciclo de reloj.
- Duplicación de Unidades Funcionales: En máquinas superescalares, se utilizan múltiples ALUs o unidades de punto flotante para que, si dos instrucciones necesitan sumar al mismo tiempo, cada una tenga su propio hardware.

14) Describa 3 diferentes causas que pueden retardar un cauce de instrucciones segmentado. ¿Qué retardo produce cada una?

Dependencias de Datos

Ocurren cuando una instrucción necesita el resultado de una instrucción anterior que todavía no ha terminado de escribirse en los registros.

- Causa: La dependencia más común es RAW (Read After Write), también llamada Dependencia Verdadera.
- Retardo: El cauce debe detener las instrucciones posteriores hasta que el dato esté disponible. Esto suele costar entre 1 y 3 ciclos de reloj, dependiendo si existe o no una unidad de Forwarding.

Dependencias de Control

Ocurren debido a las instrucciones de salto.

- Causa: El procesador no sabe cuál es la siguiente instrucción a buscar hasta que se evalúa la condición del salto.
- Retardo: Si el procesador espera a saber el resultado del salto (si se produce o no), el cauce queda vacío durante varios ciclos. Si el procesador usa predicción de saltos y falta, debe limpiar todas las instrucciones cargadas por error. Lo que puede costar de 2 a 15 ciclos en procesadores con cauces profundos (supersegmentados).

Dependencias Estructurales

Ocurren cuando dos o más instrucciones en diferentes etapas del cauce necesitan utilizar el mismo recurso físico al mismo tiempo.

- Causa: Por ejemplo, si se tiene una sola memoria para datos e instrucciones (arquitectura de Von Neumann), en un ciclo, una instrucción intenta leer un dato (MEM) mientras que otra intenta buscar una nueva instrucción (IF). Como solo hay un bus, no pueden hacerlo a la vez.
- Retardo: Se genera un ciclo de espera (STALL) obligatorio. Una de las instrucciones debe esperar a que el recurso se libere retrasando todo el flujo del programa por 1 ciclo de reloj cada vez que ocurre el conflicto.

<u>15) Explique el mecanismo de interrupción.</u>

Ciclo de reconocimiento (Hardware)

Todo inicia con una señal eléctrica fuera de la CPU:

- Solicitud: Un periférico activa la línea **INTR** del procesador o del controlador de interrupciones (PIC).
- Consulta: La CPU termina de ejecutar la instrucción actual y comprueba el estado de la línea de interrupción.
- Aceptación: Si las interrupciones están habilitadas, la CPU envía una señal de **INTA** para avisar que atenderá el pedido.

Salvado del contexto

Antes de saltar al código de la interrupción, la CPU debe asegurarse de poder volver a donde estaba:

- Hardware automático: El procesador guarda automáticamente en la pila el valor del PC y el registro de estado (PWS).
- Software (opcional): Una vez dentro del gestor de interrupción, el programado suele guardar el resto de los registros en la pila (mediante instrucciones PUSH) para evitar que la rutina de servicio los pise.

Localización de la rutina (vectorización)

Para identificar la ubicación donde debe ir la CPU:

- El periférico (o el PIC) entrega un Vector de Interrupción (número).
- La CPU usa ese número como índice de la IVT (Tabla de Vectores de Interrupción) para obtener la dirección de memoria donde comienza la ISR (Interrupt Service Routine).

Ejecución del gestor (ISR)

La CPU carga la dirección obtenida en el PC y comienza a ejecutar el código específico para atender ese periférico.

Retorno al Programa Principal

Al finalizar la rutina, se ejecuta la instrucción especial IRET (Interrupt Return).

- Esta instrucción desapila los FLAGS y el PC originales.
- El procesador recupera su estado anterior y sigue ejecutando la siguiente instrucción del programa principal como si nada hubiera pasado.

<u>16) Describa el tratamiento a realizar cuando hay múltiples fuentes de interrupciones.</u>

Existen varias estrategias para identificar la fuente de la interrupción:

Múltiples líneas de interrupción: Se dispone de una línea de interrupción por cada dispositivo. Es probable que cada línea se le conecten a varios módulos de E/S, debido a la cantidad restringida de señales de interrupción que puede manejar el CPU.

Ante múltiples interrupciones, el procesador selecciona la línea con más prioridad.

Consulta de Software (Software Polling): Se dispone de una sola línea de interrupción para los dispositivos. Cuando ocurre el pedido de interrupción, el CPU tiene que consultar a cada dispositivo para determinar quién fue el demandante.

Ante múltiples interrupciones, el orden en el que se consultan los módulos determina su prioridad.

Conexión en cadena (Daisy Chain): Todos los módulos de E/S comparten una línea común para solicitar interrupciones.

La línea de reconocimiento de interrupción se conecta encadenando los módulos uno tras otro.

Una vez enviada la confirmación de parte del CPU, el módulo que está más adelante en la conexión encadenada responderá colocando su número de identidad en el bus, el cual será utilizado como puntero a la rutina de servicio apropiada.

Ante múltiples interrupciones, el orden de los módulos en la conexión en cadena determina su prioridad.

Arbitraje de bus (vectorizada): Se dispone de una sola línea de interrupción INTR para todos los dispositivos, un controlador dedicado (PIC) provee el vector que identifica la fuente de interrupción.

Ante múltiples interrupciones, las líneas de interrupción proveen una manera de gestionar la prioridad, la línea 0 es la que mayor prioridad tiene. Sin embargo, es configurable mediante Software.

17) Describa las funciones de correspondencia entre memoria principal y caché.

La función de correspondencia determina cómo se organiza la caché, es decir, la forma en la que se van a asignar los bloques de la memoria principal en la memoria caché.

- Correspondencia directa: Consiste en hacer corresponder cada bloque de memoria principal a solo una línea posible de la caché. Esto se implementa utilizando la dirección. Cada dirección de memoria principal puede verse como dividida en tres campos:

    - Tag: Número de grupo en la memoria principal.

    - Line: Número de bloque dentro del grupo.

    - Word: Número de palabra dentro del bloque.

  Esto proporciona una correspondencia única o asignación única de cada bloque de memoria principal en la caché, lo cual quiere decir que solo existe una posibilidad donde el bloque puede existir en esta.

  Su principal desventaja es una baja tasa de aciertos y grandes pérdidas de caché, debido a que hay solo una posición concreta de caché para cada bloque dado, si un programa accede a palabras de dos bloques diferentes en la misma línea de la caché, se intercambiarán continuamente.

- Correspondencia asociativa: Permite que cada bloque de memoria principal pueda cargarse en cualquier línea de la caché.

- Correspondencia asociativa por bloques: Un bloque puede almacenarse en un conjunto restringido de lugares en la caché.

  La caché se divide en un número de conjuntos N, cada conjunto N contiene un número de líneas y un bloque corresponderá a alguna línea de un conjunto determinado. Combina lo mejor de las otras correspondencias.

18) ¿Qué es y de qué depende el paralelismo de una máquina?

Paralelismo a nivel hardware

- Paralelismo temporal (Segmentación): Se logra dividiendo la ejecución de una instrucción en etapas. Este paralelismo depende de la cantidad de etapas, cuantas más etapas (supersegmentación), más instrucciones pueden estar en vuelo simultáneamente.

- Paralelismo espacial (Superescalaridad): Se logra duplicando las unidades de ejecución. En este caso, el paralelismo depende de la cantidad de recursos físicos (ALUs, cores, etc.) disponibles para realizar tareas al mismo tiempo.

Paralelismo real

Por más que se tenga mucho hardware, se deben tener en cuenta los siguientes factores:

- Dependencias de Datos: Límite principal. Si una instrucción necesita el resultado de la anterior (dependencia verdadera), el paralelismo se detiene.
- Dependencias de Control: El paralelismo depende fuertemente de la Unidad de Predicción de Saltos. Si el procesador no sabe hacia dónde va el código, no puede cargar instrucciones por adelantado.
- Ancho de banda de la memoria: No sirve tener muchos cores o ALUs si el sistema no puede traer datos de la memoria lo suficientemente rápido para alimentarlos.

Paralelismo a nivel software

- ILP (Instruction Level Parallelism): Depende de qué tan independientes son las instrucciones entre sí en un mismo flujo. Es una medida de "qué tan paralelo" puede ser un programa.
- TLP (Thread Level Parallelism): Depende de la capacidad del sistema operativo y del código para ejecutar flujos totalmente separados (hilos) en distintos contextos de ejecución (cores o threads lógicos).

<u>19) ¿Por qué funciona un sistema de memoria basado en jerarquía?</u>

Principio de localidad

- **Localidad Temporal:** Si un dato se usó recientemente, es muy probable que se vuelva a usar pronto. Por eso se almacena en la caché.
- **Localidad Espacial:** Si se accede a un dato, es muy probable que pronto se acceda a los datos que están almacenados físicamente cerca. Por eso, cuando la caché busca un dato, trae un bloque entero de la RAM.

Compromiso de la tecnología: Es imposible crear una memoria gigante y súper rápida:

- SRAM (Caché): Extremadamente rápido pero ocupa mucho espacio físico en el chip y es muy cara.
- DRAM (RAM): Más lenta y barata, permitiendo tener gigabytes de capacidad.
- Discos: Muy lento (en relación a S/DRAM) pero masivo y económico.

Funcionamiento en cascada: La jerarquía funciona porque el sistema intenta que el dato necesario esté siempre en el nivel más alto (el más rápido):

- El procesador busca en la caché, si no lo encuentra (en cualquiera de sus niveles), luego va a la RAM.

<u>20) Describa las características funcionales del acceso directo a memoria (DMA).</u>

Independencia de la CPU: La característica principal es que el controlador de DMA puede tomar el control de los buses del sistema de manera autónoma. Mientras el DMA mueve

datos, la CPU puede estar ejecutando otras instrucciones que no requieran los buses, logrando un verdadero paralelismo.

Transferencia de bloques: A diferencia del método por interrupciones o consulta, el DMA está diseñado para mover bloques de datos completos.

El procesador solo interviene al principio, para configurar la dirección de inicio y el tamaño del bloque, y al final, para recibir la interrupción de que la tarea terminó.

Modos de operación:

- **Modo Ráfaga (Burst Mode):** El DMA toma el bus y no lo suelta hasta que termina de mover todo el bloque. Es el más rápido para el periférico, pero deja esperando al CPU.
- **Robo de Ciclo (Cycle Stealing):** El DMA toma el bus por un solo ciclo de reloj para mover un dato y lo devuelve. Es más lento para la transferencia, pero no interrumpe tanto el trabajo del procesador.
- **Modo Transparente:** El DMA solo usa los buses cuando detecta que la CPU no los está usando.

Registros de Configuración: Para funcionar, el DMA requiere que la CPu cargue los valores en sus registros internos antes de empezar:

- **Registro de Dirección de Memoria:** Destino u origen en la RAM.
- **Contador de Palabras:** Cuánto falta mover.
- **Registro de Control:** Define si es Lectura o Escritura y el modo de transferencia.

Notificación por interrupción: Una vez que el contador llega a cero, el controlador de DMA activa una señal de interrupción (INTR) hacia la CPU. De esta manera le hace saber al sistema operativo que los datos ya están listos en la memoria para ser procesados.

<u>21) ¿A qué método de interrupción lo conocemos como de "interrupciones vectorizadas"?</u>

Las interrupciones vectorizadas son aquellas donde el periférico proporciona un índice (vector) que apunta a una entrada en una tabla de memoria. Este índice permite a la CPU localizar y ejecutar la rutina de servicio correspondiente de forma directa y eficiente.

1. Petición: El dispositivo activa la línea de interrupción (INTR).
2. Reconocimiento: El CPU responde con una señal de INTA (Interrupt Acknowledge).
3. Entrega del vector: Al recibir el INTA, el dispositivo (o el PIC) coloca un número de 8 bits en el bus de datos.
4. Salto a la IVT: El CPU toma ese número y lo usa como índice para buscar en la **tabla de vectores de interrupción (IVT)**. En esa tabla está almacenada la dirección de memoria donde comienza la ISR (Rutina de Servicio de Interrupción) específica para ese dispositivo.

22) Esquematice y describa la estructura interna de un módulo de E/S.

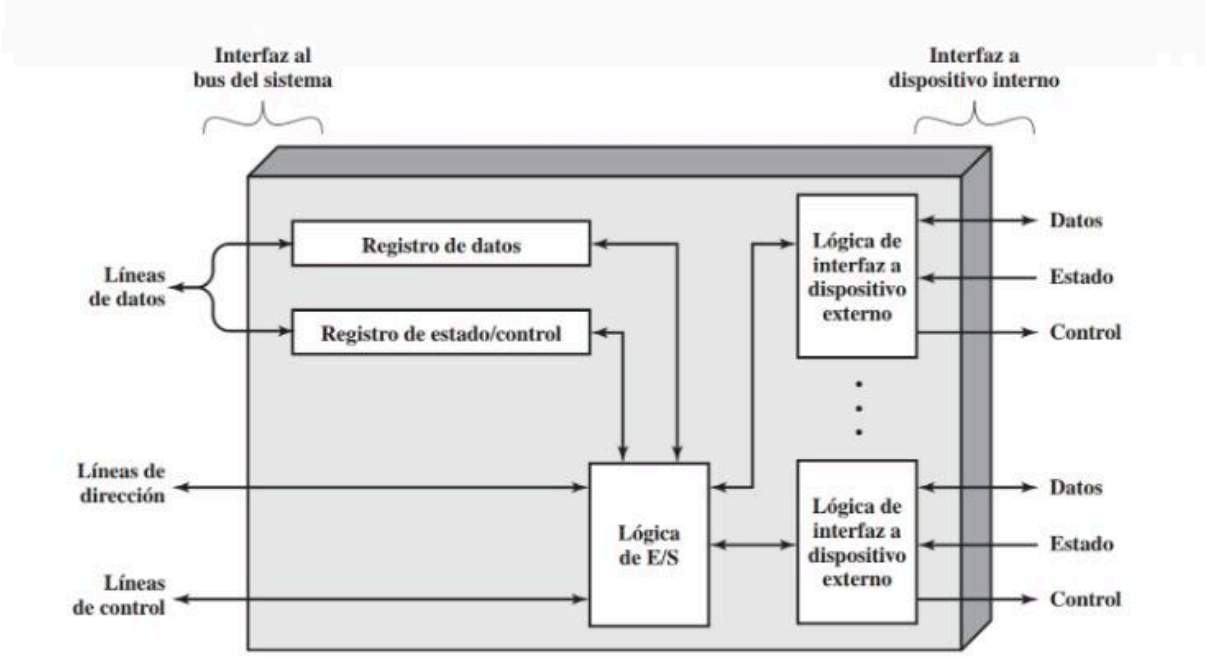


Figura 7.3. Diagrama de bloques de un módulo de E/S.

Interfaz con el Bus del Sistema: Son las conexiones que van hacia la CPU y la RAM.

- Líneas de Datos: Por acá viaja la información real que se quiere leer o escribir. Se conectan directamente a los Registros de Datos.
- Líneas de Dirección: La CPU las utiliza para seleccionar este módulo específico entre todos los que hay en el sistema.
- Líneas de Control: Transportan órdenes como “Lectura” o “Escritura”.

Registros Internos

- Registros de Datos: Funcionan como un buffer. Sirven para compensar la diferencia de velocidad: la CPU deja el dato ahí a toda velocidad y el módulo se encarga de enviarlo al periférico lentamente, o viceversa.
- Registro de Estado/Control:
    - Estado: El módulo pone los bits ahí para que el CPU los lea.
    - Control: El CPU escribe ahí para configurar cómo debe actuar el módulo.

Lógica de E/S: Cerebro interno del módulo:

- Decodifica las direcciones que vienen del CPU para saber si le están hablando a él.
- Interpreta las señales de control (READ (RD) y WRITE (WR)).
- Gestionar el flujo de datos entre los registros internos y el dispositivo externo.

- Enviar señales de interrupción a la CPU cuando sea necesario.

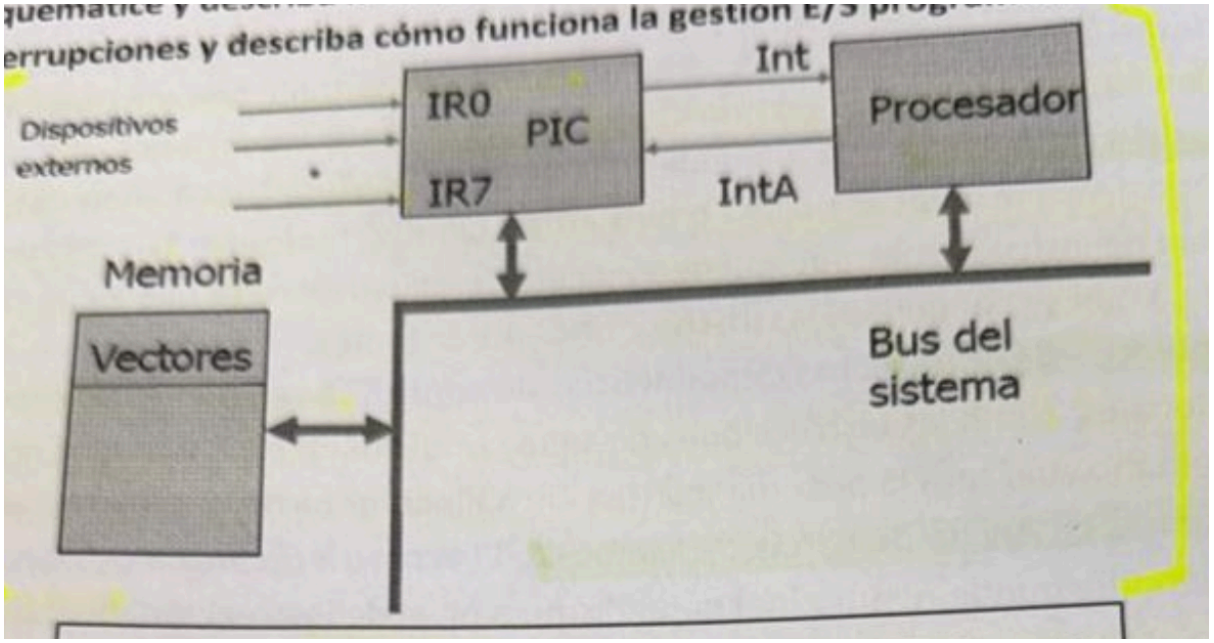
Interfaz al Dispositivo Externo: Cables físicos que van al periférico real.

- Datos: La información que entra o sale del dispositivo.
- Estado: Señales que vienen del periférico.
- Control: Señales que el módulo envía para comandar al periférico.

Ejemplo de funcionamiento del módulo de E/S:

1. El CPU pone la dirección del módulo en las líneas de dirección
2. Pone el carácter en las líneas de datos.
3. Activa la línea de control de “Escritura”.
4. La lógica de E/S captura el carácter y lo guarda en el Registro de Datos.
5. Finalmente, la lógica lo envía por la interfaz externa hacia la impresora siguiendo sus tiempos lentos.

<u>23) Esquematice y describa la estructura interna del Controlador Programable de Interrupciones (PIC).</u>



Componentes del sistema

- Dispositivos externos: Son los periféricos que generan las peticiones de interrupción Sus líneas se conectan a las entradas IR0 a IR7 del PIC.
- PIC: Recibe las interrupciones de los periféricos, decide cuál es más importante y le avisa al procesador.

- Procesador: Ejecuta el programa principal y se detiene solo cuando el PIC se lo ordena.
- Memoria (Vectores): Donde reside la Tabla de Vectores de Interrupción. Cada entrada de esta tabla contiene la dirección de memoria donde empieza la rutina de interrupción para cada dispositivo.
- Bus del sistema: Bus por donde viajan las direcciones y los datos entre todos los componentes.

Flujo de comunicación:

1. Petición: Un dispositivo externo activa una línea hacia el PIC.
2. Aviso (Int): El PIC envía una señal por la línea INT al procesador.
3. Confirmación (IntA): El procesador responde por la línea IntA.
4. Identificación: El PIC pone en el Bus de Sistema el número de vector correspondiente.
5. Búsqueda: El procesador toma ese número, va a la Memoria y busca la dirección de la rutina que tiene que ejecutar.

24) ¿Qué características tienen los procesadores superescalares?

Múltiples Unidades Funcionales (Duplicación de Hardware): Característica física más evidente y el motivo por el cual los procesadores superescalares pueden ejecutar múltiples instrucciones de manera paralela.

El procesador tiene varias ALUs, unidades de punto flotante y unidades de carga/almacenamiento.

Emisión múltiple: La Unidad de Control es capaz de buscar (fetch), decodificar y lanzar a las unidades de ejecución múltiples instrucciones simultáneamente en cada ciclo de reloj.

Ejecución fuera de orden: Para que las unidades funcionales no se queden ociosas, el procesador no sigue estrictamente el orden del programa:

- Si la instrucción A está trabada esperando un dato de memoria, pero la instrucción B y C ya tienen sus datos listos, el procesador las ejecuta primero. Esto requiere una Ventana de Instrucciones donde el hardware analiza qué puede adelantar.

Renombramiento de registros: Al ejecutar múltiples instrucciones a la vez, se generan conflictos por usar los mismos nombres de registros. El procesador usa registros ficticios adicionales para mapear estos nombres y eliminar dependencias falsas.

Si bien no es una característica exclusiva de los procesadores superescalares, es principalmente utilizada por ellos.

Predicción de saltos avanzada: Como el procesador procesa tantas instrucciones por ciclo, un error en una predicción de salto puede retrasar mucho el cauce debido a que debe

vaciarse. Por este motivo, los procesadores superescalares incluyen lógica de predicción dinámica muy compleja para mantener el flujo constante.

25) Describa las causas que pueden retardar el funcionamiento de un procesador superescalar.

Dependencias de Datos (Dependencias verdaderas): Es el freno principal. Pasa cuando una instrucción necesita el resultado de una anterior (Read After Write).

Riesgos Estructurales: Pasan cuando el hardware no es lo suficiente para la demanda de las instrucciones lanzadas en paralelo.

Dependencias de Control: Pasa cuando el procesador encuentra un salto condicional, debe predecir hacia dónde va y falla.

Falsas dependencias: Si bien el Renombramiento de Registros ayuda a mitigar este tipo de dependencias, si el procesador se queda sin registros físicos “ficticios” para asignar, debe detenerse.

- WAR (Write After Read): Escribir en un registro que todavía debe ser leído por una instrucción anterior.
- WAW (Write After Write): Dos instrucciones intentando escribir en el mismo registro.

Cuellos de botella en el retiro: Aunque las instrucciones se ejecuten fuera de orden para ganar velocidad, deben terminar en el orden original del programa para asegurar que el resultado sea correcto.

26) ¿Qué características tienen los procesadores supersegmentados?

Gran cantidad de tapas de cauce: Mientras que un procesador segmentado clásico tiene entre 5 y 7 etapas, uno supersegmentado puede tener 15, 20 o más. Esto se logra dividiendo tareas básicas en sub-etapas más granulares.

Aumento de la frecuencia de reloj (MHz/GHz): Al ser cada etapa del cauce mucho más corta y simple, el tiempo necesario para que las señales eléctricas se estabilicen es menor.

Al iniciar una etapa cada medio ciclo de reloj, aumentar la velocidad duplica la cantidad de instrucciones que entran al cauce.

Latencia de instrucción más alta: Aunque el procesador termina instrucciones más rápido (tiene mayor throughput), una sola instrucción tarda más ciclos de reloj en completarse desde que entra hasta que sale.

La cantidad de ciclos que tarda en ejecutarse una etapa no cambia, si existen 20 etapas, la instrucción tardará 20 ciclos. Tarda lo mismo en cantidad de etapas, pero menos en cantidad de tiempo.

Alta penalización ante fallos

- Si hay un error en la predicción de un salto, el procesador tiene que vaciar un cauce de muchos más niveles.

- Se pierden muchos ciclos de reloj para volver a llenar el cauce, lo que puede anular por completo la ganancia de velocidad que se obtuvo al subir la frecuencia.

27) Describa las características que pueden retardar el funcionamiento de un procesador supersegmentado.

La causas que retardan a un procesador supersegmentado son similares a las de un cauce normal, pero el impacto se multiplica debido a la mayor profundidad del pipeline. Al tener tantas etapas, cualquier interrupción se vuelve mucho más costosa en términos de ciclos perdidos.

Gran penalización por saltos: Si el procesador predice mal el salto, debe vaciar todas las instrucciones que ya habían entrado al cauce por el camino incorrecto.

Aumento en el retraso por dependencias: En supersegmentación, las dependencias de datos causan paradas más largas.

Si una instrucción necesita un resultado de la instrucción inmediatamente anterior, y esa instrucción está dividida en 10 sub-etapas de ejecución, el dato tardará muchos más ciclos de reloj en estar disponible.

Mayor latencia de memoria: El procesador supersegmentado corre a frecuencias de reloj altísimas (ciclos muy cortos).

La RAM no escala a la misma velocidad. Por lo tanto, una espera por un dato que no está en la caché representa muchísimos más ciclos de reloj perdidos en un procesador supersegmentado que en uno convencional.

Desequilibrio entre etapas: Al dividir las tareas en etapas pequeñas, es muy difícil lograr que todas tarden exactamente lo mismo.

Si una sub-etapa es levemente más lenta que las demás, todo el procesador debe funcionar a la velocidad de esa etapa más lenta.

28) ¿Cuándo, por qué, para qué y cómo se utiliza una de las denominadas interrupciones por software?

Se utilizan cuando un programa de usuario necesita realizar una tarea que solo el Sistema Operativo tiene permiso de hacer (modo Kernel).

Se utilizan por seguridad y estabilidad. Si cualquier programa pudiera escribir directamente en el disco duro o acceder a la memoria de otro proceso, el sistema sería un caos y muy inseguro.

El procesador utiliza estas interrupciones para forzar un cambio de contexto controlado, asegurando que el programa solo haga lo que el Sistema Operativo le permite.

A diferencia de las interrupciones por hardware, que son señales eléctricas, estas se disparan mediante una instrucción específica en el código, ej INT 21h, que activa un cambio de privilegio en el CPU y salta a una rutina específica en el Kernel.

Otro ejemplo es INT 0, que el procesador dispara automáticamente cuando detecta una división por cero. Aunque no es una llamada al sistema que el programador pide, funciona igual. El hardware detiene el programa y salta a una rutina del kernel para evitar que el CPU se bloquee por un cálculo imposible.

29) Describa los posibles modos de ubicación de los módulos de E/S.

E/S mapeada en memoria (Memory Mapped I/O): El procesador no distingue entre una posición de la memoria RAM y un registro de un módulo de E/S. Comparten el mismo espacio de direccionamiento.

Se reserva un rango de direcciones de la memoria principal para los periféricos.

Esto permite la utilización de las mismas instrucciones que para la memoria (MOV, LOAD, STORE), lo que simplifica el repertorio de instrucciones del CPU.

Por otro lado, tiene la desventaja de que el espacio disponible para la RAM se reduce, ya que se pierden direcciones que ahora pertenecen a los módulos de E/S.

E/S independiente o aislada (Isolated I/O): El sistema tiene dos espacios de direcciones totalmente separados, uno para la RAM y otro exclusivo para los módulos de E/S.

El procesador usa una señal de control adicional para indicar si la dirección que está poniendo en el bus es para la memoria o para un periférico.

Elimina la desventaja anterior, deja de robar espacio de direccionamiento a la memoria RAM, pero requiere de instrucciones específicas del CPU (como IN y OUT) para comunicarse con los periféricos.

30) Describa el funcionamiento de la gestión de E/S programada mediante interrupciones.

La CPU interviene directamente en la transferencia de cada unidad de información con el módulo.

Cada vez que el módulo esté listo (o haya completado una transferencia), avisa al CPU con un pedido de interrupción.

Ahora, el procesador solo inicia la transferencia al recibir el pedido de interrupción del periférico, y puede continuar realizando algún trabajo útil.

1) El dispositivo de E/S envía una señal de interrupción al procesador.
2) El procesador completa la ejecución de la instrucción en curso antes de responder a la interrupción.
3) El procesador comprueba si hay interrupciones, determina que hay una, y envía una señal de reconocimiento al dispositivo que originó la interrupción.
4) El procesador guarda en la pila la información necesaria para continuar con el programa en curso en el punto en que se interrumpió (PSW (Program Status Word, registro/conjunto de datos que representa el estado actual del CPU) y PC).

5) Después, el procesador carga el nuevo PC en función de la interrupción. El control se transfiere al programa de gestión de interrupción, que realiza estas tareas:

    a) Guarda los contenidos de los registros del procesador, ya que podrían ser modificados.

    b) Procesa la interrupción. El procesador ejecuta la instrucción de entrada o salida para mover **una única unidad de información** entre el módulo de E/S y la memoria.

    c) Este proceso se repite por cada dato a transferir. Si el módulo debe enviar un bloque de información, enviará una interrupción por cada palabra, obligando a la CPU a realizar todo el ciclo de salvado y restauración de contexto una y otra vez.

    d) Cuando el procesamiento de la interrupción ha terminado, los valores de los registros almacenados se recuperan de la pila y se vuelven a almacenar en los registros.

    e) Se restaura el PC y el PSW, para continuar en donde se dejó el programa interrumpido.